

UNIVERSITÉ DE KINSHASA
FACULTÉ DES SCIENCES ET TECHNOLOGIES
MENTION MATHÉMATIQUES, STATISTIQUE ET
INFORMATIQUE



departement.mathinfo@unikin.ac.cd
B.P. 190 KINSHASA XI

CONCEPTION ET IMPLÉMENTATION
D'UN SYSTÈME DE SIGNATURE
NUMÉRIQUE SÉCURISÉ AVEC
TRAÇABILITÉ

Réalisé par les étudiants :

BASENGA LOLEKA SAMUELLA
HISSEIN MAHAMAT DRYA
KISENGO EXCELLENCE
MAZINA MASINDA JEREMIE
TIBISA MAGAKILA BIENHEUREUSE

Professeur : Prof. KASENGEDIA
Collaborateur : CT.JUNIOR KANINGINI

Domaine : Sciences et Technologies
Promotion : M1 INFORMATIQUE

Année Académique 2025 – 2026

Table des matières

1	Introduction	3
2	Problématique	4
3	Objectifs du Projet	4
3.1	Objectif général	4
3.2	Objectifs spécifiques	4
4	Technologies utilisées	4
4.1	Environnement de développement	5
4.2	Choix technologiques	6
5	Architecture du système	6
6	Architecture réseau (Cisco)	7
6.1	Structure réseau	7
6.2	Segmentation VLAN	7
6.3	Schéma logique	7
6.4	Implémentation sous Cisco Packet Tracer	8
6.5	Extrait de configuration	8
7	Sécurité implémentée	9
7.1	Mécanismes cryptographiques	9
7.1.1	Algorithme RSA (2048 bits)	9
7.1.2	Fonction de hachage SHA-256	10
7.2	Authentification et gestion des accès	10
7.2.1	JSON Web Token (JWT)	10
7.2.2	Gestion des rôles	10
7.3	Sécurité applicative	10
7.3.1	Middleware Helmet	10
7.3.2	Limitation de débit (Rate Limiting)	10
7.3.3	Configuration CORS sécurisée	11
7.4	Sécurité réseau	11
7.5	Analyse de sécurité avec Kali Linux	11
7.5.1	Scan des ports avec Nmap	11
7.5.2	Audit Web avec Nikto	11
7.5.3	Résultats observés	11
7.6	Synthèse	11
8	Tests de sécurité avec Kali Linux	12
8.1	Reconnaissance avec Nmap	12
8.2	Audit de vulnérabilités avec Nikto	12
8.3	Vérification du Rate Limiting avec Curl	13
8.4	Synthèse et bilan de l'audit de sécurité	13
9	Présentation des Interfaces	14

10 Fonctionnement du processus de signature	18
11 Code source	18
12 Synthèse du projet	19
12.1 Avantages du système	19
12.2 Limites et perspectives	19
13 Perspectives	19
14 Conclusion	20

1 Introduction

À l'ère de la transformation numérique, la dématérialisation des documents est devenue un enjeu stratégique pour les organisations modernes. Cependant, cette transition vers le "zéro papier" s'accompagne de cybermenaces croissantes, rendant la sécurisation des échanges électroniques plus critique que jamais pour la survie des systèmes d'information.

L'objectif principal de ce projet est de concevoir et de déployer un système robuste de signature numérique de documents avec traçabilité, capable de répondre aux exigences fondamentales de la confiance numérique. Ce dispositif repose sur le principe de l'authenticité, qui garantit l'identité du signataire et l'origine du document, ainsi que sur l'intégrité, assurant que le contenu n'a subi aucune altération depuis sa validation. Enfin, il intègre la dimension de non-répudiation, rendant impossible la contestation de l'acte de signature par son auteur.

Cette étude propose une approche holistique et multidisciplinaire. Elle intègre non seulement le développement d'une application web sécurisée basée sur une architecture 3-tiers, mais s'appuie également sur une infrastructure réseau segmentée en zones de confiance. Pour valider la viabilité de cette solution, un audit de sécurité offensif a été mené, permettant de confronter le système aux réalités des menaces actuelles.

2 Problématique

Le passage au numérique expose les documents à des risques critiques d'usurpation d'identité et de falsification. Contrairement au format papier, un fichier numérique peut être modifié ou copié sans laisser de trace visible, ce qui fragilise la confiance dans les échanges électroniques. Le problème central de ce travail peut se résumer à la question suivante : Comment garantir techniquement l'authenticité et l'intégrité d'un document numérique tout en protégeant l'infrastructure qui l'héberge contre les cyberattaques ?

Pour répondre à ce défi, notre étude se concentre sur deux axes :

- L'utilisation de la cryptographie pour sécuriser l'acte de signature.
- La mise en place d'une architecture réseau isolée (DMZ) pour sécuriser le stockage et le traitement des données.

3 Objectifs du Projet

3.1 Objectif général

Le but principal de ce travail est de mettre en place une plateforme sécurisée permettant la signature numérique et le suivi des documents, en garantissant une étanchéité totale entre les services grâce à une architecture réseau avancée.

3.2 Objectifs spécifiques

Pour atteindre cet objectif, nous avons défini les axes de développement suivants :

- **Ingénierie Applicative** : Développer une application web moderne (Architecture 3-tiers) intégrant une API REST sécurisée.
- **Implémentation Cryptographique** : Mettre en œuvre les algorithmes de pointe tels que **RSA** (pour la signature) et **SHA-256** (pour le hachage).
- **Infrastructure Réseau** : Concevoir et simuler, sous *Cisco Packet Tracer*, une architecture segmentée par VLANs avec isolation des services en zone DMZ.
- **Validation et Audit** : Réaliser un audit de sécurité offensif à l'aide de la distribution **Kali Linux** (Nmap, Nikto, Curl) pour valider la robustesse du système face aux attaques courantes.

4 Technologies utilisées

Dans le cadre de ce projet, plusieurs technologies logicielles et outils de sécurité ont été utilisés afin de concevoir une plateforme complète de signature numérique sécurisée. Ces technologies couvrent aussi bien le développement applicatif, la sécurité cryptographique, l'administration réseau que les tests d'audit de sécurité.

4.1 Environnement de développement

Le système a été développé selon une architecture moderne basée sur une séparation claire entre la couche frontend, backend et la base de données. Cette approche permet une meilleure maintenabilité, une évolutivité du système ainsi qu'une amélioration des performances globales.

TABLE 1 – Technologies principales utilisées dans le projet

Technologie	Catégorie	Rôle dans le projet
React.js	Frontend	Développement de l'interface utilisateur dynamique et interactive.
Node.js	Backend	Exécution du serveur applicatif côté backend.
Express.js	Framework Backend	Création de l'API REST sécurisée et gestion des routes HTTP.
MongoDB	Base de données NoSQL	Stockage des documents, signatures et informations utilisateurs.
JWT (JSON Web Token)	Authentification	Gestion sécurisée des sessions et authentification des utilisateurs.
RSA 2048 bits	Cryptographie asymétrique	Génération et vérification des signatures numériques.
SHA-256	Fonction de hachage	Garantie de l'intégrité des documents numériques.
NGINX	Reverse Proxy	Routage des requêtes et sécurisation des communications HTTPS.
Helmet	Sécurité HTTP	Protection des en-têtes HTTP contre plusieurs attaques web.
Express Rate Limit	Protection API	Limitation des requêtes pour prévenir les attaques DoS et brute force.
CORS	Sécurité Web	Contrôle des domaines autorisés à accéder à l'API.
Cisco Packet Tracer	Simulation réseau	Conception et simulation de l'architecture VLAN et DMZ.
Kali Linux	Audit de sécurité	Réalisation des tests de sécurité offensifs.
Nmap	Scanner réseau	Analyse des ports ouverts et détection des services réseau.
Nikto	Audit Web	Détection des vulnérabilités sur le serveur web.
Curl	Test HTTP	Vérification des requêtes API et du mécanisme Rate Limiting.
Git / GitHub	Gestion de version	Suivi et gestion du code source du projet.
Visual Studio Code	Éditeur de code	Développement et maintenance du projet.

4.2 Choix technologiques

Le choix des technologies utilisées dans ce projet repose sur plusieurs critères, notamment la sécurité, la performance, la modularité et la compatibilité avec les architectures modernes orientées services.

Le framework React.js a été retenu pour sa capacité à construire des interfaces utilisateur réactives et performantes. Du côté serveur, Node.js associé à Express.js offre une excellente gestion des communications temps réel et des API REST sécurisées.

MongoDB a été privilégié comme système de gestion de base de données grâce à sa flexibilité dans le stockage des données non structurées et sa capacité de montée en charge.

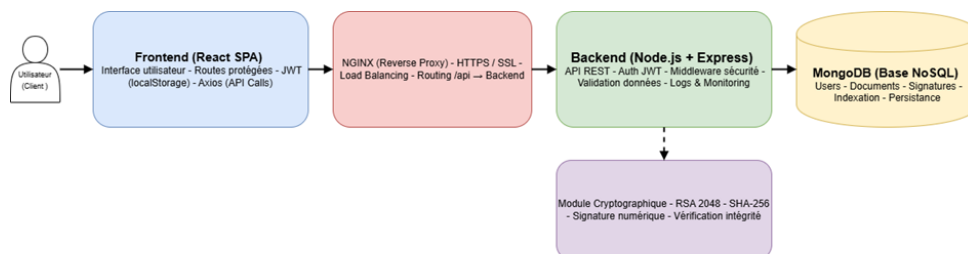
Concernant la sécurité, l'utilisation des algorithmes RSA et SHA-256 permet d'assurer respectivement l'authenticité des signatures numériques et l'intégrité des documents. Les mécanismes JWT, Helmet et Rate Limiting renforcent également la sécurité applicative contre les attaques courantes.

Enfin, Cisco Packet Tracer et Kali Linux ont permis de simuler un environnement réseau sécurisé et de réaliser différents tests d'audit afin de valider la robustesse globale du système.

5 Architecture du système

Le système repose sur une architecture 3-tiers :

- Frontend : React
- Backend : Node.js + Express
- Base de données : MongoDB



Architecture sécurisée avec reverse proxy et module cryptographique

FIGURE 1 – Architecture globale

Cette architecture repose sur un modèle 3-tiers enrichi par un reverse proxy NGINX assurant la sécurisation des communications via HTTPS ainsi que la gestion du routage des requêtes. Le frontend développé en React communique avec une API REST Node.js sécurisée par JWT. Le backend intègre des mécanismes de validation, d'authentification et de journalisation. La sécurité des données est renforcée par l'utilisation de la cryptographie RSA pour la signature numérique et SHA-256 pour garantir l'intégrité des documents. Les données sont stockées dans une base MongoDB NoSQL optimisée pour la performance et la scalabilité.

6 Architecture réseau (Cisco)

6.1 Structure réseau

L'application backend est déployée sur un serveur situé dans une zone DMZ (VLAN 30) afin d'isoler les services critiques. Les utilisateurs situés dans le VLAN 10 et les administrateurs dans le VLAN 20 accèdent à l'application via le routage inter-VLAN configuré sur le routeur. La communication est testée via des requêtes ICMP (ping) et HTTP, garantissant la disponibilité du service dans un environnement réseau segmenté et sécurisé.

6.2 Segmentation VLAN

Pour isoler les services critiques, l'application est déployée dans une zone démilitarisée (DMZ). Le réseau est segmenté en trois VLANs distincts :

- **VLAN 10 (USERS)** : Accès restreint pour les utilisateurs finaux.
- **VLAN 20 (ADMINS)** : Accès total pour l'administration du système.
- **VLAN 30 (DMZ)** : Hébergement des serveurs Backend et Frontend.

6.3 Schéma logique

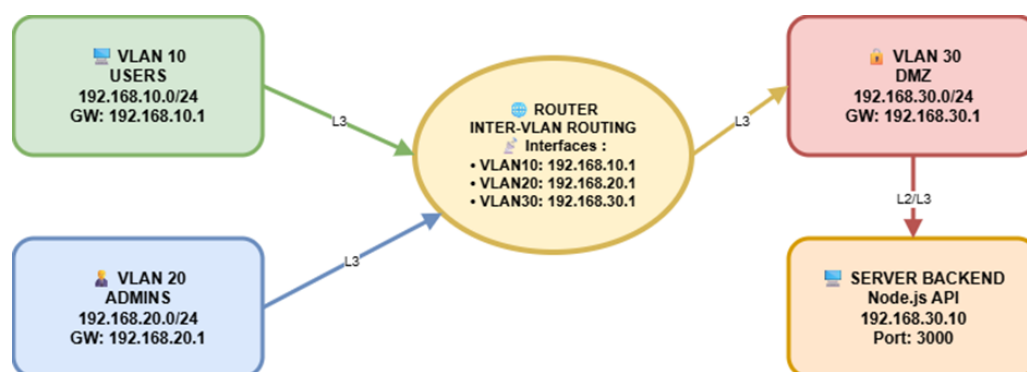


FIGURE 2 – Architecture logique du réseau segmenté par VLAN

VLAN	Nom	Adresse
10	USERS	192.168.10.0/24
20	ADMINS	192.168.20.0/24
30	DMZ	192.168.30.0/24

Le routage inter-VLAN est assuré par un routeur (Router-on-a-Stick).
Le serveur backend est placé dans la DMZ afin de :

- isoler les services critiques ;
- limiter les attaques ;
- renforcer la sécurité globale du système.

6.4 Implémentation sous Cisco Packet Tracer

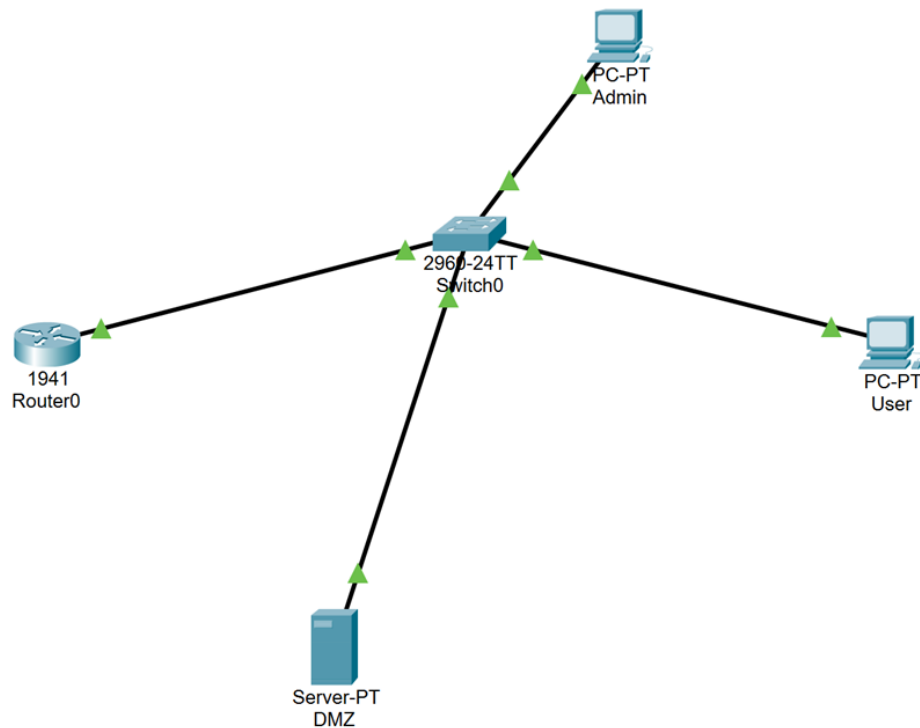


FIGURE 3 – Simulation du réseau sous Cisco Packet Tracer

6.5 Extrait de configuration

```
interface g0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0

interface g0/0.30
encapsulation dot1Q 30
ip address 192.168.30.1 255.255.255.0
```

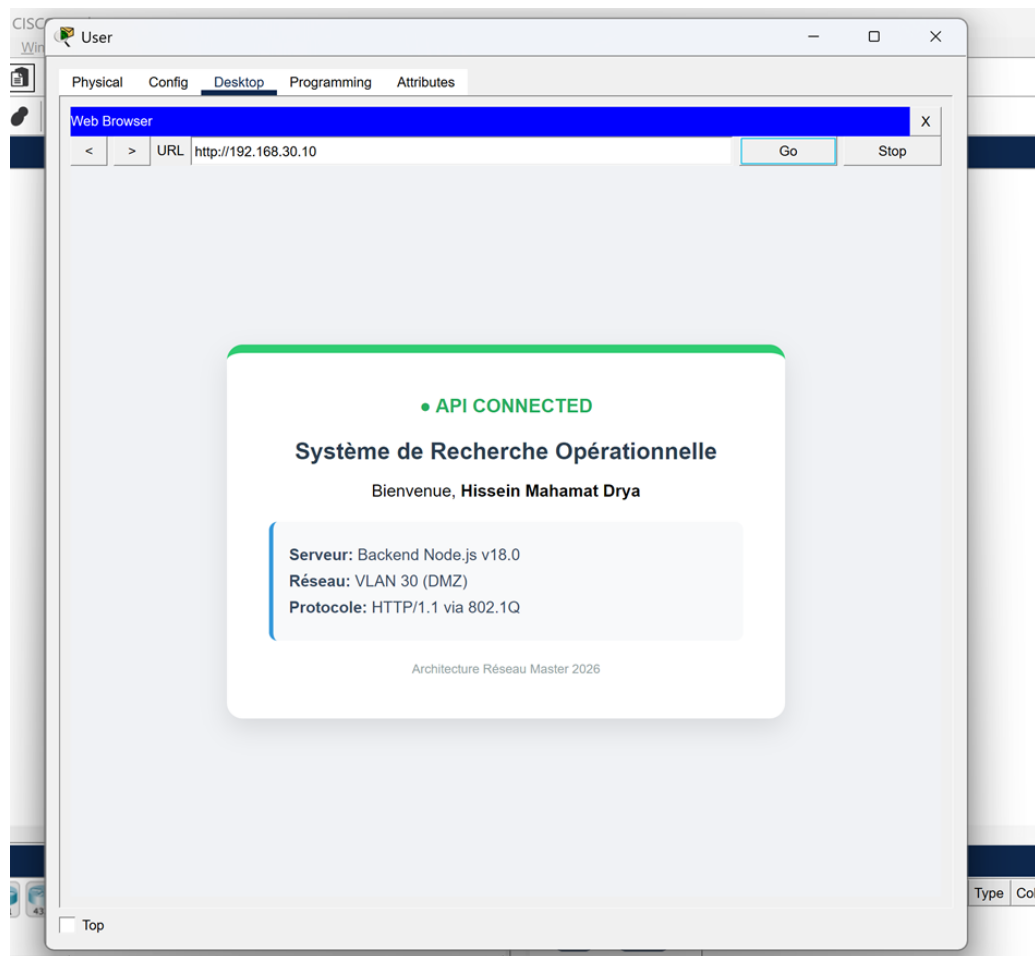


FIGURE 4 – Accès à l'API depuis le réseau

7 Sécurité implémentée

La sécurité constitue un axe central de ce projet. Elle repose sur une approche multicouche intégrant des mécanismes cryptographiques, des contrôles d'accès, ainsi que des protections applicatives et réseau.

7.1 Mécanismes cryptographiques

7.1.1 Algorithme RSA (2048 bits)

Le système utilise l'algorithme asymétrique RSA avec une taille de clé de 2048 bits afin de garantir un niveau de sécurité élevé.

- Génération d'une paire de clés (clé publique / clé privée).
- La clé privée est utilisée pour signer les documents.
- La clé publique permet de vérifier l'authenticité de la signature.

Ce mécanisme assure :

- **Authenticité** : identification fiable du signataire.
- **Non-répudiation** : impossibilité pour le signataire de nier son engagement.

7.1.2 Fonction de hachage SHA-256

Avant la signature, chaque document est transformé en une empreinte numérique (hash) à l'aide de l'algorithme SHA-256.

- Production d'un condensé unique représentant le document.
- Toute modification, même minime, entraîne un changement du hash.

Ce mécanisme garantit :

- **Intégrité des données** : détection immédiate de toute altération du document.

7.2 Authentification et gestion des accès

7.2.1 JSON Web Token (JWT)

L'authentification des utilisateurs est assurée à l'aide de jetons JWT (JSON Web Token).

- Génération d'un token après validation des identifiants.
- Transmission sécurisée dans les requêtes HTTP.
- Durée d'expiration configurée (7 jours).

Les JWT permettent :

- Une gestion des sessions sans état (*stateless*).
- Une réduction de la charge côté serveur.
- Une sécurisation efficace des routes de l'API.

7.2.2 Gestion des rôles

Le système implémente un contrôle d'accès basé sur les rôles :

- **Utilisateur** : accès limité aux fonctionnalités de signature et de consultation.
- **Administrateur** : accès complet incluant la gestion des utilisateurs et l'audit du système.

7.3 Sécurité applicative

7.3.1 Middleware Helmet

Le module Helmet est utilisé pour sécuriser les en-têtes HTTP. Il permet notamment :

- La protection contre les attaques XSS (Cross-Site Scripting).
- La prévention du clickjacking via l'en-tête X-Frame-Options.
- L'activation de politiques de sécurité avancées (Content Security Policy).

7.3.2 Limitation de débit (Rate Limiting)

Un mécanisme de limitation du nombre de requêtes a été mis en place afin de prévenir les attaques par déni de service (DoS).

- Limite fixée à 100 requêtes par intervalle de 15 minutes.
- Blocage automatique en cas de dépassement (code HTTP 429).

Ce dispositif permet de protéger le serveur contre :

- Les attaques par force brute.
- Les abus d'utilisation de l'API.

7.3.3 Configuration CORS sécurisée

Le système limite strictement les origines autorisées à accéder à l'API :

- Autorisation uniquement du domaine du frontend.
- Blocage des requêtes provenant de sources non approuvées.

7.4 Sécurité réseau

La sécurité réseau repose sur une segmentation logique via des VLANs :

- **VLAN 10** : réseau des utilisateurs.
- **VLAN 20** : réseau des administrateurs.
- **VLAN 30** : zone DMZ hébergeant les serveurs applicatifs.

Cette segmentation permet de limiter la surface d'attaque et d'isoler les services critiques.

Le serveur backend est isolé dans la DMZ, ce qui permet de limiter l'exposition directe et de réduire considérablement la surface d'attaque.

7.5 Analyse de sécurité avec Kali Linux

Une phase de test de sécurité a été réalisée à l'aide des outils Kali Linux.

7.5.1 Scan des ports avec Nmap

- Identification des ports ouverts (ex : 5000 pour l'API).
- Détection des services exposés et des versions logicielles.

7.5.2 Audit Web avec Nikto

L'outil Nikto a permis d'identifier :

- Les en-têtes HTTP manquants ou mal configurés.
- Le comportement du serveur face aux requêtes malveillantes.

7.5.3 Résultats observés

- Présence effective des mécanismes de sécurité (CSP, XSS protection).
- Activation réelle du Rate Limiting (erreur 429 observée lors des tests).
- API robuste contre les accès abusifs.

7.6 Synthèse

L'approche de sécurité adoptée combine :

- Cryptographie asymétrique forte.
- Authentification centralisée et sécurisée.
- Protection applicative contre les vulnérabilités courantes.
- Isolation réseau stricte par segmentation.
- Validation par tests d'intrusion.

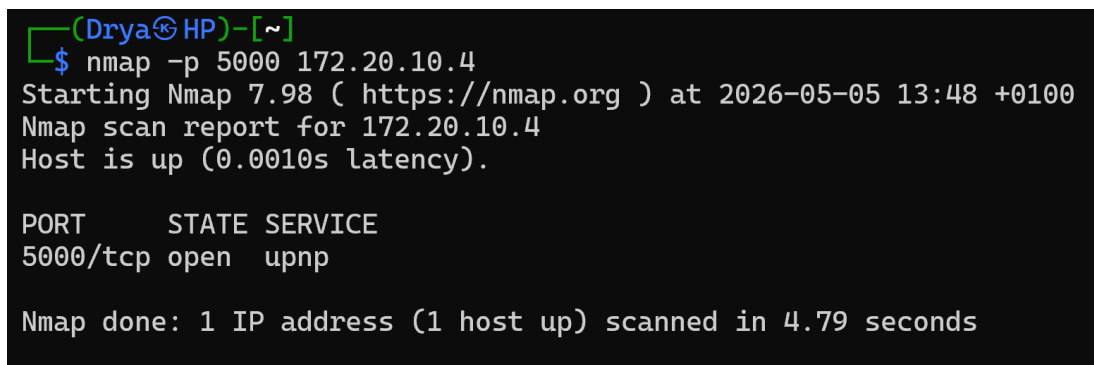
8 Tests de sécurité avec Kali Linux

L'audit de sécurité a été réalisé depuis une machine attaquante sous Kali Linux pour valider la robustesse de l'API déployée dans la DMZ.

8.1 Reconnaissance avec Nmap

Le scan de ports permet de vérifier que seul le port nécessaire au service est exposé.

```
nmap -p 5000 172.20.10.4
```



```
(Dryas HP)~  
$ nmap -p 5000 172.20.10.4  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-05 13:48 +0100  
Nmap scan report for 172.20.10.4  
Host is up (0.0010s latency).  
  
PORT      STATE SERVICE  
5000/tcp  open  upnp  
  
Nmap done: 1 IP address (1 host up) scanned in 4.79 seconds
```

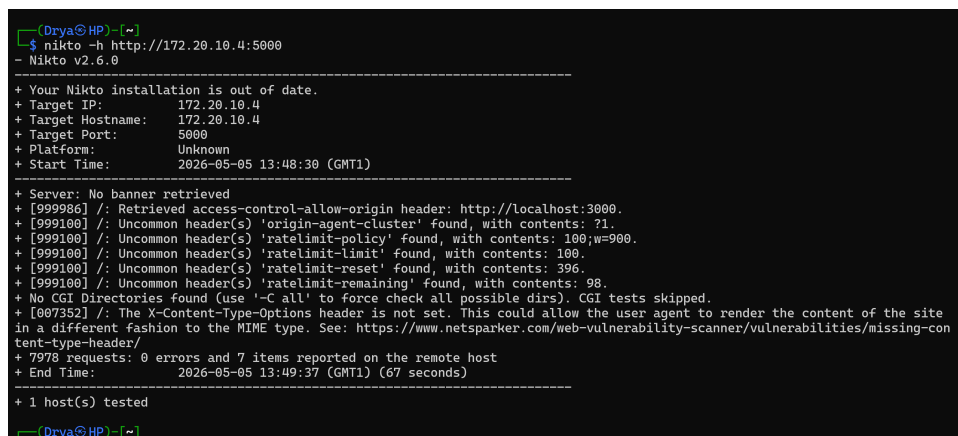
FIGURE 5 – Capture d'écran du scan Nmap confirmant l'ouverture du port 5000

Le résultat confirme que le port **5000 (TCP)** est ouvert, permettant ainsi l'accès aux services de signature numérique.

8.2 Audit de vulnérabilités avec Nikto

L'outil Nikto a été utilisé pour analyser les en-têtes de sécurité et les configurations serveur.

```
nikto -h http://172.20.10.4:5000
```



```
(Dryas HP)~  
$ nikto -h http://172.20.10.4:5000  
- Nikto v2.6.0  
-----  
+ Your Nikto installation is out of date.  
+ Target IP: 172.20.10.4  
+ Target Hostname: 172.20.10.4  
+ Target Port: 5000  
+ Platform: Unknown  
+ Start Time: 2026-05-05 13:48:30 (GMT1)  
-----  
+ Server: No banner retrieved  
+ [999986] /: Retrieved access-control-allow-origin header: http://localhost:3000.  
+ [999100] /: Uncommon header(s) 'origin-agent-cluster' found, with contents: 71.  
+ [999100] /: Uncommon header(s) 'ratelimit-policy' found, with contents: 100;w=900.  
+ [999100] /: Uncommon header(s) 'ratelimit-limit' found, with contents: 100.  
+ [999100] /: Uncommon header(s) 'ratelimit-reset' found, with contents: 396.  
+ [999100] /: Uncommon header(s) 'ratelimit-remaining' found, with contents: 98.  
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.  
+ [007352] /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/  
+ 7978 requests: 0 errors and 7 items reported on the remote host  
+ End Time: 2026-05-05 13:49:37 (GMT1) (67 seconds)  
-----  
+ 1 host(s) tested  
(Dryas HP)~
```

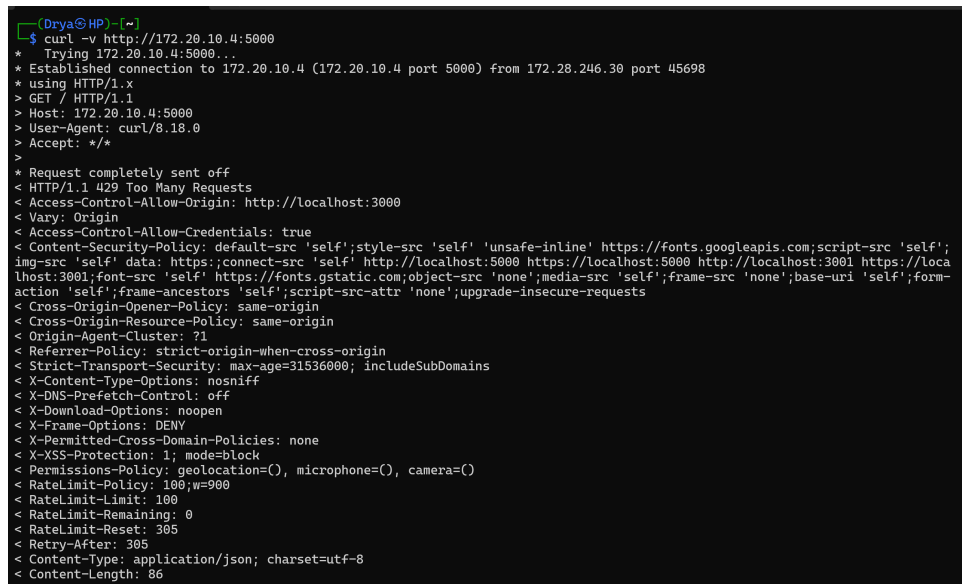
FIGURE 6 – Analyse des en-têtes HTTP et de la sécurité avec Nikto

L'analyse démontre que les protections injectées par le middleware *Helmet* sont bien actives.

8.3 Vérification du Rate Limiting avec Curl

Afin de tester la résistance aux attaques par déni de service (DoS) ou brute force, nous avons simulé des requêtes répétitives avec *Curl*.

```
curl -v http://172.20.10.4:5000
```



```
[Dreya@HP] ~$ curl -v http://172.20.10.4:5000
* Trying 172.20.10.4:5000...
* Established connection to 172.20.10.4 (172.20.10.4 port 5000) from 172.28.246.30 port 45698
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 172.20.10.4:5000
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 429 Too Many Requests
< Access-Control-Allow-Origin: http://localhost:3000
< Vary: Origin
< Access-Control-Allow-Credentials: true
< Content-Security-Policy: default-src 'self';style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;script-src 'self';
img-src 'self' data: https;connect-src 'self' http://localhost:5000 https://localhost:5000 http://localhost:3001 https://loca
lhost:3001;font-src 'self' https://fonts.gstatic.com;object-src 'none';media-src 'self';frame-src 'none';base-uri 'self';form-
action 'self';frame-ancestors 'self';script-src-attr 'none';upgrade-insecure-requests
< Cross-Origin-Opener-Policy: same-origin
< Cross-Origin-Resource-Policy: same-origin
< Origin-Agent-Cluster: ?1
< Referrer-Policy: strict-origin-when-cross-origin
< Strict-Transport-Security: max-age=31536000; includeSubDomains
< X-Content-Type-Options: nosniff
< X-DNS-Prefetch-Control: off
< X-Download-Options: noopen
< X-Frame-Options: DENY
< X-Permitted-Cross-Domain-Policies: none
< X-XSS-Protection: 1; mode=block
< Permissions-Policy: geolocation=(), microphone=(), camera=()
< RateLimit-Policy: 100;w=900
< RateLimit-Limit: 100
< RateLimit-Remaining: 0
< RateLimit-Reset: 305
< Retry-After: 305
< Content-Type: application/json; charset=utf-8
< Content-Length: 86
```

FIGURE 7 – Validation du mécanisme de limitation de débit (HTTP 429)

Le serveur renvoie un code **429 Too Many Requests**, prouvant que le mécanisme de *Rate Limiting* protège efficacement l'API contre les abus.

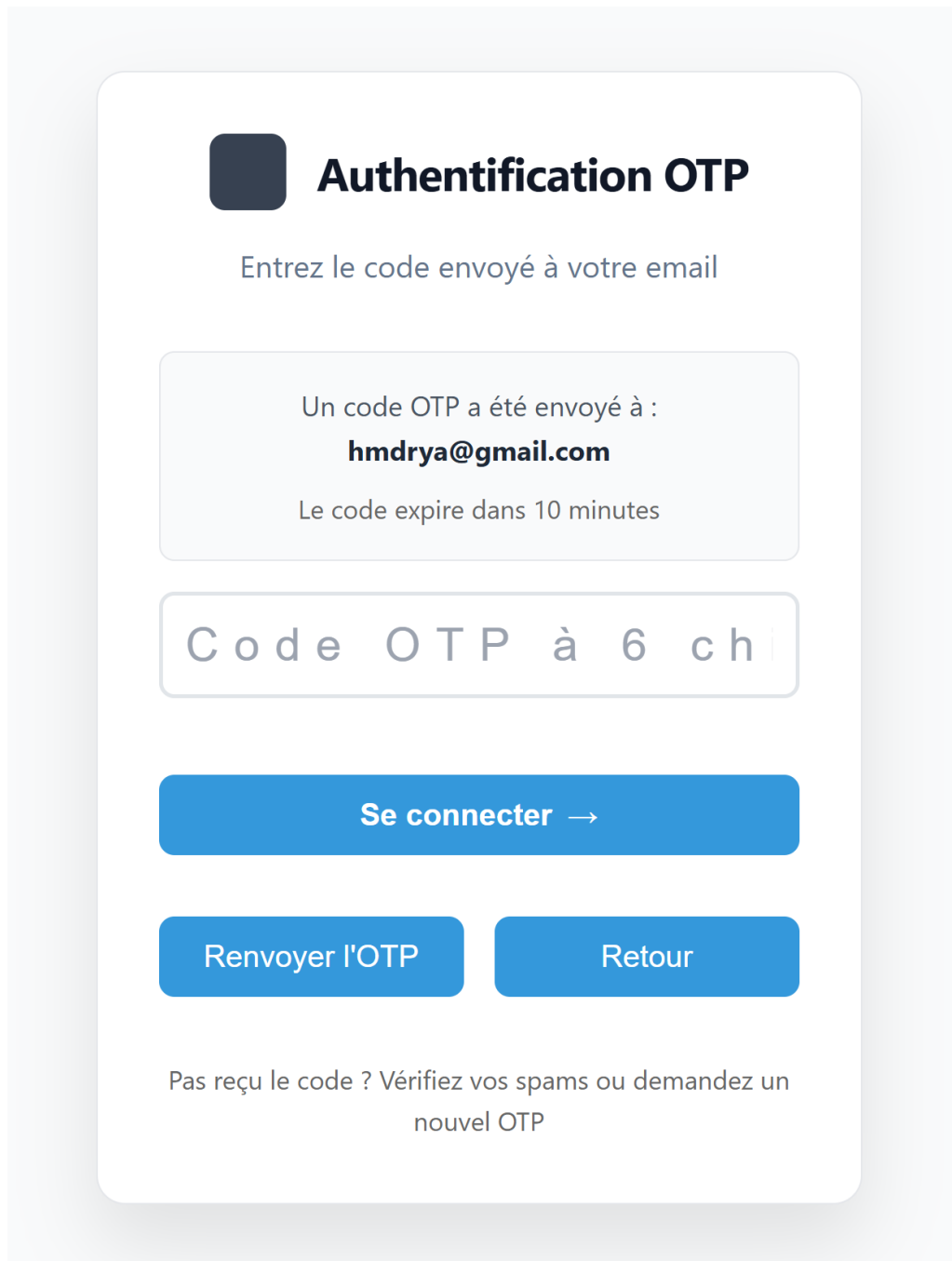
8.4 Synthèse et bilan de l'audit de sécurité

L'audit de sécurité réalisé à l'aide des outils Nmap, Nikto et Curl a permis de valider l'intégrité de l'architecture réseau du projet. Les tests confirment que le backend est correctement isolé et protégé par des politiques de sécurité modernes.

L'implémentation de Helmet.js assure une protection robuste contre les attaques de type XSS et Clickjacking grâce à des en-têtes CSP et X-Frame-Options configurés de manière restrictive. De plus, la mise en place d'un middleware de Rate Limiting a été éprouvée avec succès, limitant ainsi l'exposition du serveur aux tentatives de force brute et aux scans automatisés. Cette base sécurisée garantit que le processus de Signature Numérique, cœur de notre application, s'exécute dans un environnement contrôlé et résilient face aux menaces courantes du Web.

9 Présentation des Interfaces

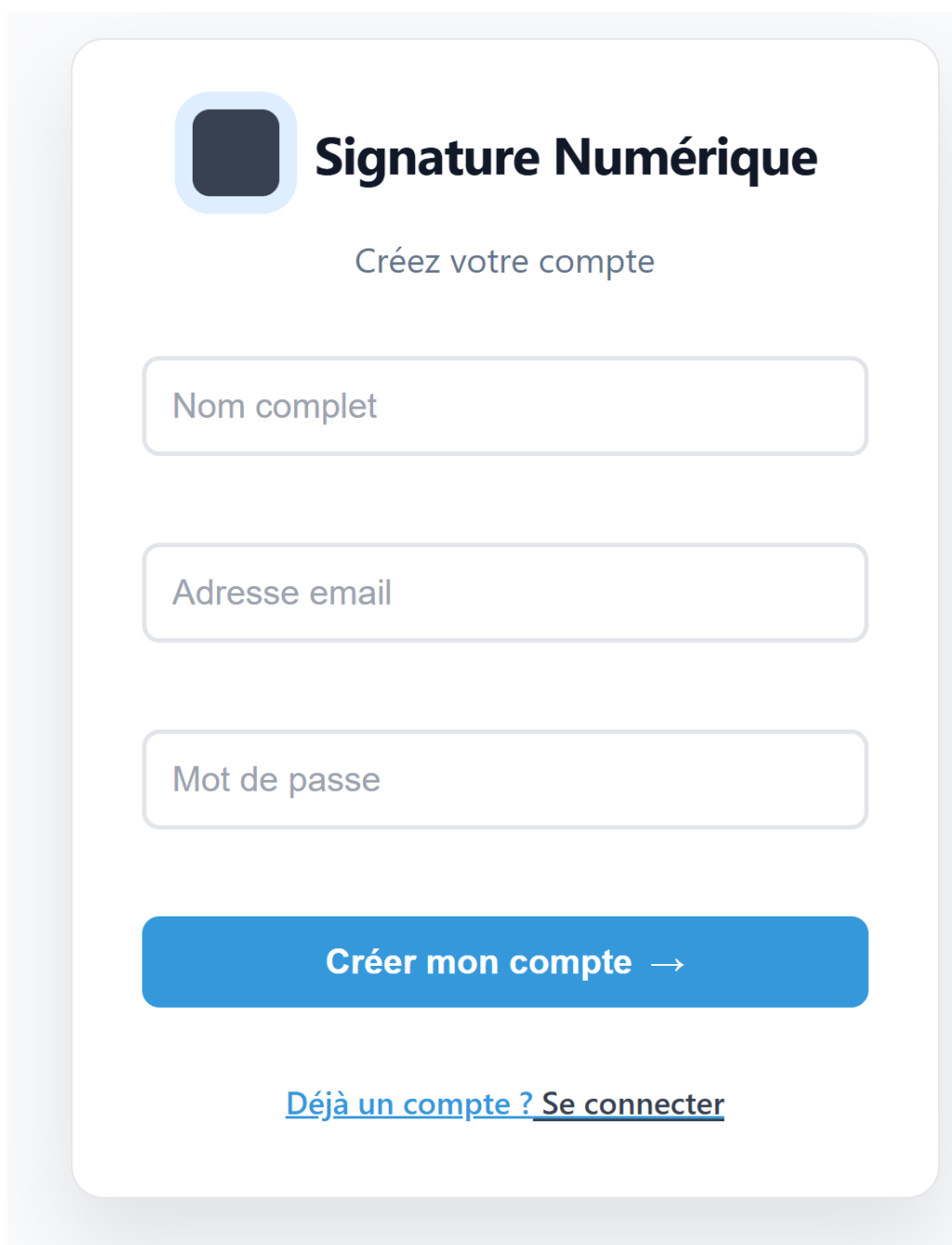
Cette section présente l'aspect visuel de l'application développée, illustrant le parcours utilisateur depuis l'authentification jusqu'à la vérification des documents.



The image shows a web interface for OTP authentication. At the top, there is a dark blue square icon followed by the title "Authentication OTP" in bold. Below the title, the instruction "Entrez le code envoyé à votre email" is displayed. A light gray box contains the text "Un code OTP a été envoyé à : hmdrya@gmail.com" and "Le code expire dans 10 minutes". Below this, a text input field is labeled "Code OTP à 6 ch". There are three blue buttons: "Se connecter →", "Renvoyer l'OTP", and "Retour". At the bottom, a link says "Pas reçu le code ? Vérifiez vos spams ou demandez un nouvel OTP".

FIGURE 8 – Interface de connexion (Login) sécurisée

L'accès à la plateforme est protégé par un formulaire d'authentification lié à une gestion de jetons JWT (JSON Web Tokens) pour assurer la persistance des sessions de manière sécurisée.



The image shows a registration interface for 'Signature Numérique'. It features a dark blue square icon with a light blue border. Below the icon is the title 'Signature Numérique' in bold. Underneath the title is the instruction 'Créez votre compte'. There are three input fields: 'Nom complet', 'Adresse email', and 'Mot de passe'. Below these fields is a blue button with the text 'Créer mon compte →'. At the bottom, there is a link that says 'Déjà un compte ? Se connecter'.

Signature Numérique

Créez votre compte

Nom complet

Adresse email

Mot de passe

Créer mon compte →

[Déjà un compte ? Se connecter](#)

FIGURE 9 – Interface d’inscription

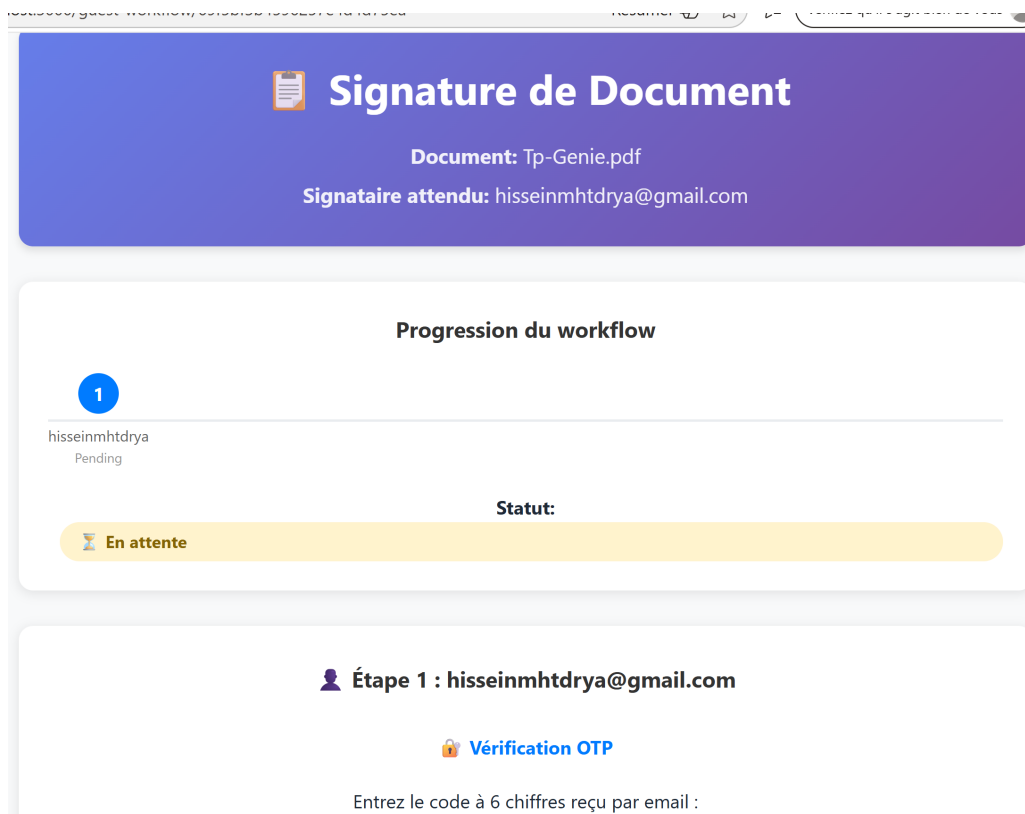


FIGURE 10 – Interface de vérification et de validation d'intégrité

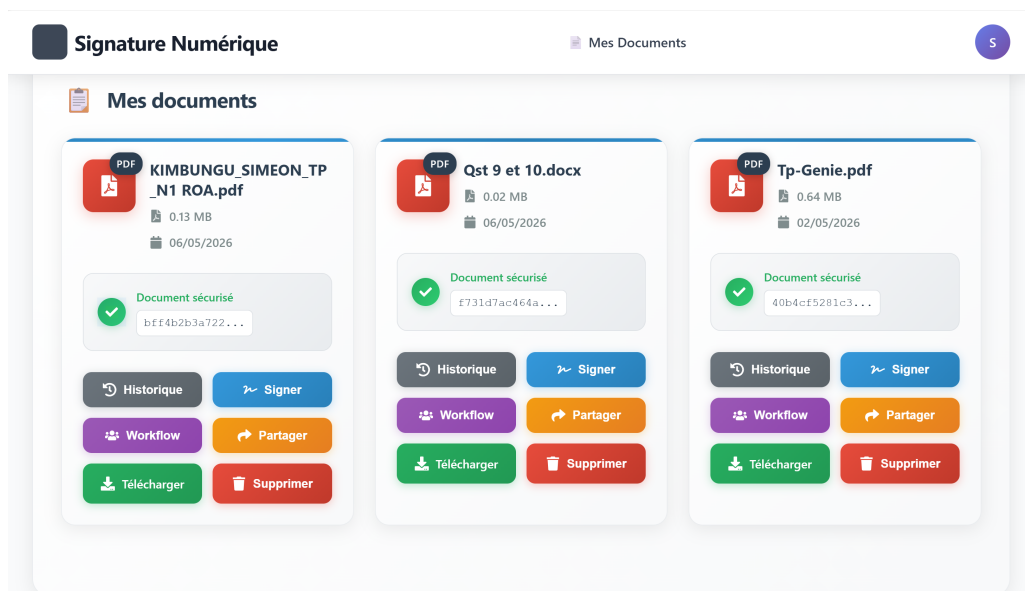


FIGURE 11 – Espace Utilisateur / Tableau de bord

Le tableau de bord permet à l'utilisateur de visualiser l'état de ses documents et d'accéder aux fonctionnalités de téléchargement vers le serveur situé dans la DMZ.

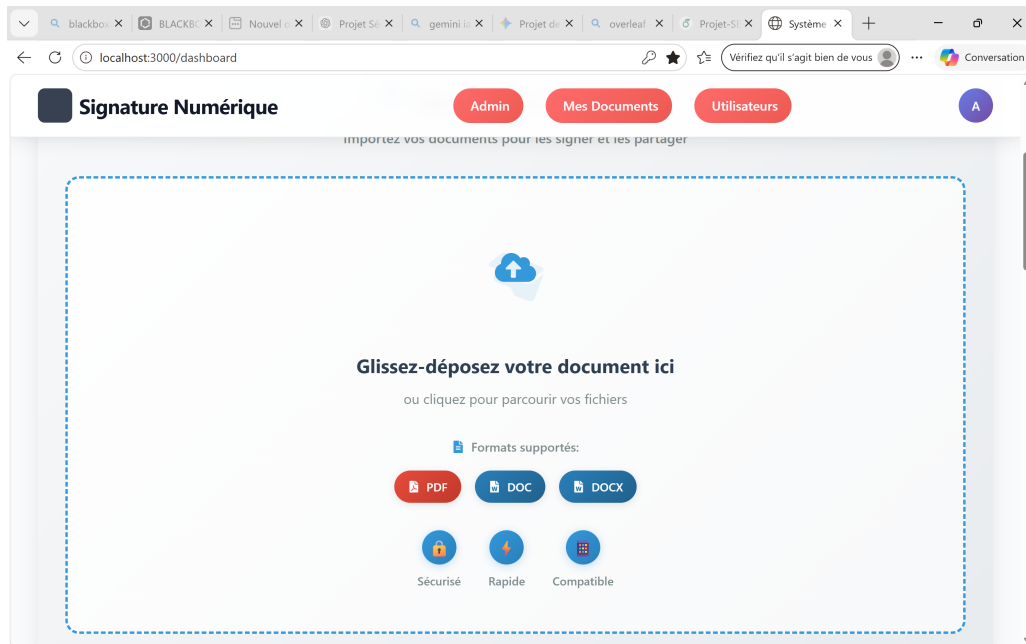


FIGURE 12 – Ajoout du documents

Cette interface permet de soumettre un document. Lors de cette étape, le système génère l’empreinte numérique et applique la signature RSA avant le stockage en base de données.

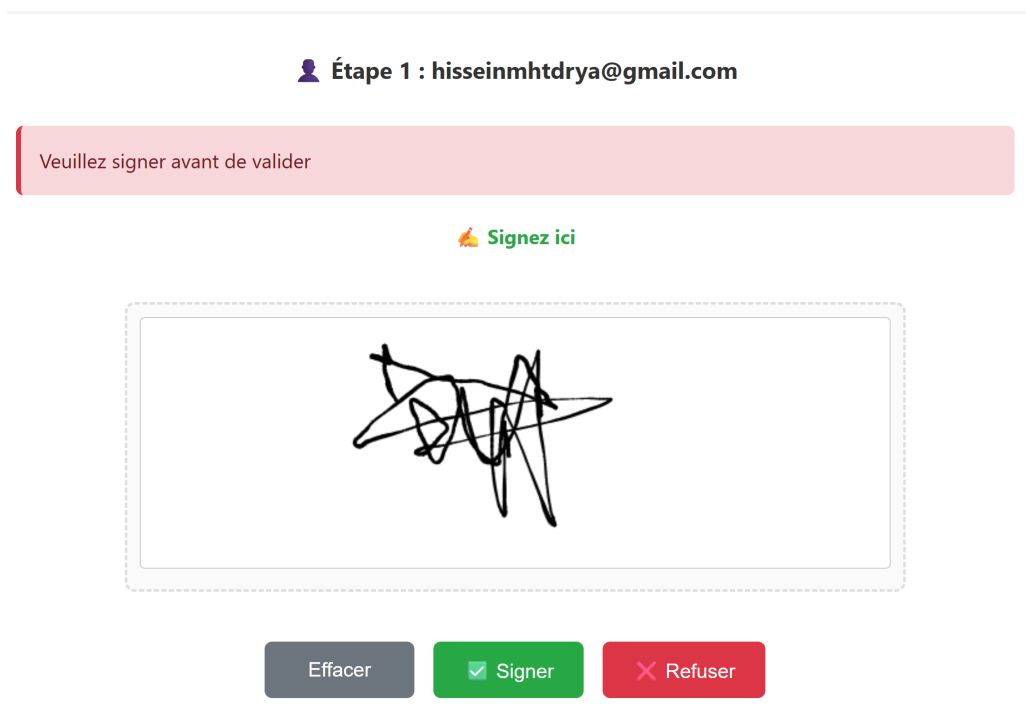


FIGURE 13 – Validation du processus de signature par workflow

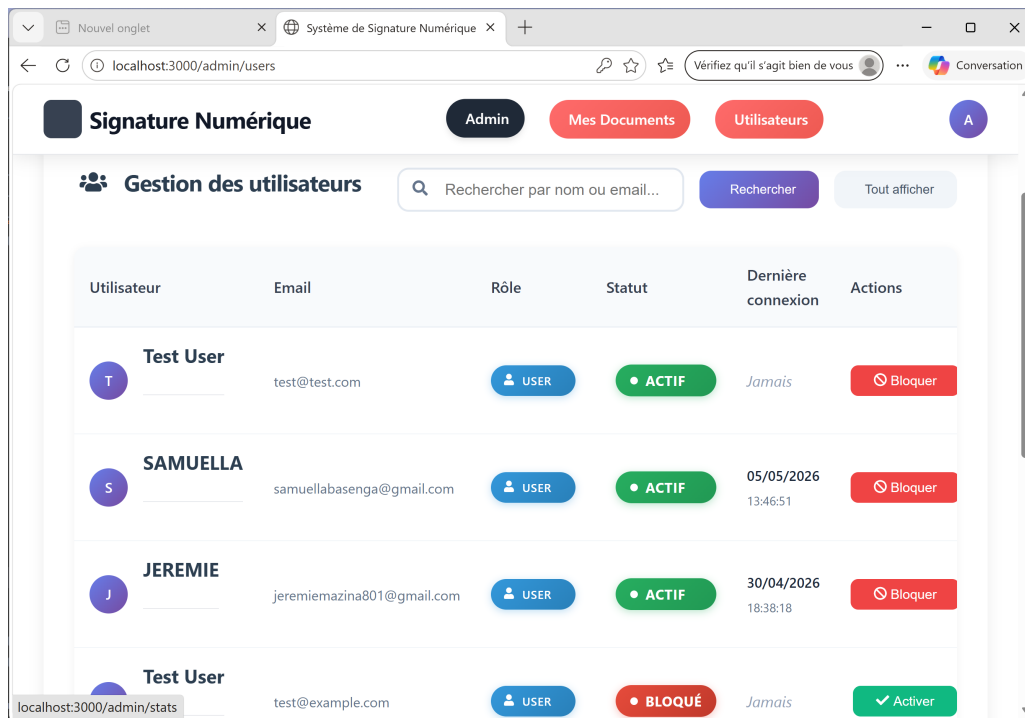


FIGURE 14 – Interface de gestion des utilisateurs (Panel Admin)

10 Fonctionnement du processus de signature

Le workflow de l'application suit une logique rigoureuse pour garantir l'intégrité :

1. **Upload** : L'utilisateur télécharge le document original sur la plateforme.
2. **Hachage** : Calcul de l'empreinte unique du document via **SHA-256**.
3. **Chiffrement** : L'empreinte est chiffrée avec la **clé privée RSA** de l'utilisateur (génération de la signature).
4. **Stockage** : Sauvegarde du document, de la signature et des métadonnées dans **MongoDB**.
5. **Vérification** : Comparaison du hash recalculé avec le hash déchiffré via la clé publique pour confirmer l'authenticité.

11 Code source

Voici le lien de notre Application :

<https://github.com/HisseinMhtDrya/SIGNATURE-NUMERIQUE>

12 Synthèse du projet

12.1 Avantages du système

- **Sécurité robuste** : Combinaison de la cryptographie asymétrique et d'une infrastructure réseau isolée (DMZ).
- **Traçabilité totale** : Chaque document est lié à une identité numérique vérifiable.
- **Scalabilité** : L'utilisation de Node.js et MongoDB permet de supporter une montée en charge importante.

12.2 Limites et perspectives

- **Infrastructure** : L'absence d'une autorité de certification (PKI) réelle limite l'usage à un environnement interne ou de test.
- **Déploiement** : Le protocole HTTPS n'est actuellement simulé que localement ; un certificat SSL (Let's Encrypt) serait nécessaire pour une mise en production.

13 Perspectives

Le système développé dans le cadre de ce projet permet d'assurer la signature numérique sécurisée des documents tout en garantissant leur intégrité et leur traçabilité. Les résultats obtenus montrent que la solution répond aux principaux besoins de sécurité et de contrôle des accès.

Cependant, certaines améliorations peuvent être envisagées afin d'optimiser davantage le système. Parmi les perspectives possibles :

- l'intégration d'une infrastructure PKI pour la gestion réelle des certificats numériques ;
- le déploiement complet du protocole HTTPS avec des certificats SSL/TLS en environnement de production ;
- l'ajout d'une authentification multi-facteurs pour renforcer la sécurité des comptes utilisateurs ;
- l'intégration d'un système de notification et de journalisation avancée des activités ;
- l'hébergement de la plateforme dans un environnement Cloud afin d'améliorer la disponibilité et la scalabilité du système.

Ces améliorations permettraient de renforcer davantage la fiabilité, la sécurité et les performances globales de la plateforme.

14 Conclusion

Ce travail de Master a permis de concevoir et de valider un système de signature numérique complet. L'intégration réussie entre le développement d'une application sécurisée et une architecture réseau segmentée sous Cisco Packet Tracer démontre l'importance d'une approche de *sécurité par la conception* (Security by Design). subsection*Perspectives Bien que le système soit fonctionnel, il ouvre la voie à plusieurs améliorations, notamment l'intégration d'une véritable infrastructure de gestion de clés (PKI) pour la certification des identités, ainsi que l'utilisation de la technologie *Blockchain* pour renforcer l'immutabilité des journaux de traçabilité.

Références

- [1] Rivest, R., Shamir, A., Adleman, L., *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*, 1978.
- [2] NIST, *Digital Signature Standard (DSS)*, 2013.
- [3] IETF, *JSON Web Token (JWT)*, RFC 7519, 2015.
- [4] Node.js Documentation, <https://nodejs.org>
- [5] React Documentation, <https://react.dev>
- [6] MongoDB Documentation, <https://www.mongodb.com>
- [7] Kali Linux Documentation, <https://www.kali.org/docs/>
- [8] Nmap Documentation, <https://nmap.org>
- [9] Nikto Web Scanner, <https://cirt.net/Nikto2>
- [10] OWASP Foundation, *OWASP Top 10 Web Application Security Risks*, <https://owasp.org>